press Ctrl-V again, at which point it will evaluate, print, and resume the loop:

```
coffee> CTRL-V

------> 'CoffeeScript!'   # Read
.......
.......
....... CTRL-V            # Eval
'CoffeeScript!'           # Print
coffee>                   # Loop
```

Now that you're familiar with the REPL, any time you are working with a single-line snippet of CoffeeScript, you can enter it into the REPL and see it evaluated:

```
'CoffeeScript!'
# 'CoffeeScript!'
```

When you see a snippet of CoffeeScript that requires the multiline mode, press Ctrl-V first and then type or paste it in, and finally press Ctrl-V again to see it evaluated.

Although the REPL is fun, and it will often be your companion as you learn CoffeeScript, you didn't come here for a lesson on how to use your keyboard. No, you came to learn about CoffeeScript, how to use it, and what it means. To begin, you want to know where you are and how you got here. How *did* you get here? The answer starts with a small historical detour, beginning with JavaScript.

1.3. JavaScript

To understand CoffeeScript and how it relates to JavaScript, you first need to know about some other languages that influenced JavaScript. Programming language influences can come in many forms, but the ones of significance in your understanding of CoffeeScript and JavaScript are the ones that led to the style, semantics, and syntax. The first influence for JavaScript in these three areas (and your starting place) is the C programming language and a humble little character called the *curly brace*.